TECHNICAL ASSESSMENT PRESENTATION | 5-MINUTE WALKTHROUGH

Slotbook: Enterprise Scheduling Engine

PostgreSQL Concurrency, Mathematical Availability & Resilient Booking Lifecycle

CANDIDATE / AUTHOR	TEST COVERAGE	CORE TECH STACK
Tirumalesh Kommavarapu	**71 Unit & E2E Tests (100% Pass)**	**Next.js 14, TypeScript, PostgreSQL**

SPEAKER SCRIPT (0:00 - 0:40) — Introduction & Context:

“Welcome everyone. Today I’m presenting Slotbook, a high-reliability appointment scheduling engine built with Next.js 14, TypeScript, and PostgreSQL. Scheduling applications face deceptive complexity: double-booking under race conditions, timezone mismatches across DST boundaries, and holding states that freeze calendars. In this presentation, I’ll demonstrate how we solved these using mathematical availability intervals, PostgreSQL exclusion constraints, and transactional outbox guarantees.”

Architectural Evolution & Core Principles

Transitioning from a prototype to a production-grade booking engine

Legacy Prototype Architecture

- **Application-level checks**: `SELECT WHERE start = ?` suffered from race conditions under concurrent requests.
- **Implicit Timezones**: Floating local dates caused DST offsets and host/guest time mismatches.
- **Direct Email Dispatch**: Network latency or SMTP timeouts failed bookings or left locks hung.
- **Fragmented State**: Confirmed bookings, holds, and cancelled slots lived in unconstrained states.

Production Hardened Architecture

- **PostgreSQL GiST Exclusions**: Database-enforced impossibility of overlapping active reservations.
- **Canonical UTC & IANA Zones**: Availability calculated in host zone; hold/booking stored in UTC timestamps.
- **Transactional Outbox Pattern**: Emails staged atomically with DB commit; background daemon dispatches.
- **Idempotency & Hold Leases**: Hold leases auto-expire via DB clock; API requests enforce idempotency keys.

SPEAKER SCRIPT (0:40 - 1:20) — Architecture Evolution:

“When assessing the initial application, we found the classic architectural pitfall: availability and conflict checks lived solely in application logic. Under concurrent traffic, application checks suffer from TOCTOU race conditions. We shifted the source of truth to PostgreSQL, enforcing physical exclusion at the database engine level, coupled with an asynchronous notification outbox for deterministic sub-150ms booking latency.”

PostgreSQL Concurrency & Exclusion Constraints

Mathematical guarantees against double-booking without distributed locks

```
- PostgreSQL btree_gist Exclusion Constraint
ALTER TABLE active_reservations
ADD CONSTRAINT no_overlapping_reservations
EXCLUDE USING gist (
host_id WITH =,
buffered_range WITH &&
);
```

Hold Expiration & Capacity Management

5-Minute Hold TTL: Holds expire authoritatively using PostgreSQL `clock_timestamp()`. No redis TTL or client clock drift can hold slots hostage.

Group Event Serialization: For multi-seat events (`capacity > 1`), booking transactions execute: `SELECT id FROM event_types WHERE id = $1 FOR UPDATE` guaranteeing exact atomic headcount updates.

Exclusion Mechanics:

- `buffered_range` is a `TSTZRANGE` spanning $[T_{\text{start}} - B_{\text{before}}, T_{\text{end}} + B_{\text{after}})$.
- Overlapping requests trigger DB error `23P01` (exclusion_violation).
- Translated cleanly by `BookingService` to `409 Conflict`.

SPEAKER SCRIPT (1:20 - 2:05) — Concurrency Model:

“Here is our core database defense: the `btree_gist` exclusion constraint. Instead of pessimistic row locks across multiple tables, we compute `buffered_range`—a timestamp range including both before and after buffer times. The `&&` overlap operator ensures PostgreSQL rejects any intersecting time slice at kernel level. Our automated tests fire 10 simultaneous parallel booking requests for the exact same millisecond: exactly one transaction succeeds (200 OK) while the remaining 9 instantly receive 409 Conflict.”

Availability Engine & Timezone Mathematics

Interval intersection, buffer step calculations, and cross-timezone projection

1. Interval Math

Base working hours are loaded per weekday in host IANA timezone. Date overrides take precedence.

Busy intervals (confirmed bookings + active holds) $A_{\text{net}} = W$ are subtracted from availability intervals: $(B \cup H)$

2. Buffer Cadence

Meeting slot iteration uses strict buffer step math: $\text{Step} = \text{Duration} + B_{\text{before}} + B_{\text{after}}$

A 30m appointment with 10m before & 10m after buffers steps at 50-minute increments, preventing back-to-back overlap.

3. Timezone Projection

Host schedules are anchored to host timezone (e.g., `Asia/Kolkata`).

Candidate slots are converted to UTC for database comparison, and rendered into guest's browser timezone (e.g. `America/New_York`) without DST distortion.

SPEAKER SCRIPT (2:05 - 2:50) — Availability & Timezones:

"Availability generation is a pure mathematical set operation. We take the host's standard weekly rules and date overrides in their configured IANA timezone, slice out busy reservations including buffer periods, and step forward by Duration plus Buffers. Notice the timezone projection: host availability is evaluated strictly in the host's local zone, stored in UTC, and projected into the guest's local timezone. A meeting at 9:00 AM IST renders as 10:30 PM EST the previous night with full DST awareness."

Transaction Integrity & Resilient Lifecycle

Transactional outbox pattern, idempotent APIs, and state machine transitions

The Transactional Outbox Pattern

Dual-Write Problem Eliminated: In standard systems, booking commits to SQL, but email dispatch fails due to network glitch—leaving state inconsistent.

Atomic Staging: `notification_outbox` record is inserted inside the same PostgreSQL transaction as the booking reservation.

Asynchronous Background Worker: A dedicated polling daemon with exponential backoff processes queued notifications, ensuring guaranteed at-least-once delivery with host and guest formatted local times.

Idempotency & Lifecycle States

- **Idempotency Keys:** Client-provided `Idempotency-Key` headers are hashed and cached with response payloads in PostgreSQL, preventing duplicate charges or duplicate slots on network retry.
- **State Transitions:**
  - `HELD` -> `CONFIRMED`
  - `HELD` -> `EXPIRED` (Auto TTL)
  - `CONFIRMED` -> `CANCELLED`
  - `CONFIRMED` -> `RESCHEDULED`

SPEAKER SCRIPT (2:50 - 3:35) — Lifecycle & Notifications:

“To ensure zero side-effect failures, we implemented the Transactional Outbox pattern. When a booking confirms, the reservation and notification payload commit atomically in the same database transaction. If SMTP experiences an outage, the booking succeeds, and the outbox processor retries with exponential backoff. Furthermore, all mutating endpoints support idempotency keys to ensure network retransmissions never create phantom bookings.”

Verification Strategy & Test Evidence

Rigorous automated test suites covering edge cases, race conditions, and UX flows

Automated Test Results (71 Passed)

Test Suite	Tests	Status
concurrency.test.ts	3 tests	PASS (100%)
scheduling_and_availability.test.ts	18 tests	PASS (100%)
critical_fixes.test.ts	12 tests	PASS (100%)
booking_lifecycle.test.ts	14 tests	PASS (100%)
auth_and_session.test.ts	10 tests	PASS (100%)
outbox_and_notifications.test.ts	8 tests	PASS (100%)
api_contracts.test.ts	6 tests	PASS (100%)

Key Edge Cases Covered

- **Simultaneous Overbooking:** 10 threads hitting the exact same slot concurrently.
- **Hold Lease Expiry:** Reservation holds releasing exactly at $T + 300\,\text{s}$.
- **Buffer Collision:** Meeting at 10:00 with 15m buffer blocking candidate at 10:30.
- **Date Override Precedence:** Single-day holiday override replacing recurring weekly rules.
- **Cancellation Lifecycle:** Dashboard counter decrements and cancelled appointments move to history tab.

SPEAKER SCRIPT (3:35 - 4:15) — Verification & Test Evidence:

“Quality is proven through empirical test evidence. Our Jest and Vitest test suites include 71 automated tests across 7 suites—all passing. We simulate high-concurrency race conditions, boundary transitions across midnight UTC, buffer collision between adjacent events, hold expiry reclamation, and atomic outbox processing. When a guest cancels an appointment, the cancellation synchronously reflects on the host dashboard while preserving historical audit logs.”

Production Deployment & Zero-Friction Setup

One-command containerized infrastructure with automated migration & health checks

Turnkey Docker Architecture

Multi-Container Compose Stack:

- `slotbook-db`: PostgreSQL 16 Alpine with `btree_gist` preloaded, persistent volume mount, and native health check.
- `slotbook-app`: Multi-stage Next.js 14 container with Alpine Node 20 runtime.

Self-Bootstrapping Entrypoint: `docker-entrypoint.sh` blocks until database health check passes, automatically executes migrations, seeds demo host data, and boots the HTTP server.

Single-Command Launch on Brand New PC

```
git clone https://github.com/tiruk007/slotbooking.git
cd slotbooking
cp .env.example .env
docker compose up –build
```

Zero host prerequisites required other than Docker Desktop and Git. Health check URL: `http://localhost:3000/api/health` returns status `200 OK` with database connection confirmation.

SPEAKER SCRIPT (4:15 - 4:45) — DevOps & Deployment:

“A robust architecture must be effortless to deploy. We designed the Docker Compose topology to be zero-prerequisite. On a brand-new laptop, a reviewer clones the repository, runs `docker compose up --build`, and the entrypoint script handles database connection pooling, running PostgreSQL migrations, seeding initial host credentials, and starting the web service. It is fully self-contained and reproducible.”

Retrospective, Trade-offs & Future Horizons

Reflections on design choices, operational lessons, and enterprise scaling

Key Trade-offs Made

- **DB Exclusion vs Redis**: Chose PostgreSQL GiST exclusions over Redis distributed locks to eliminate dual-system consistency risks and lock-split brains.
- **Outbox Polling vs Kafka**: Chose transactional relational outbox table for low operational overhead.

Critical Lessons

- Always validate schemas bidirectionally: frontend state matching Zod types avoids API rejections.
- Never trust application clocks for reservation leases: database `clock_timestamp()` is authoritative.

Enterprise Roadmap

- Multi-host collective scheduling (Round Robin & Collective availability).
- Bidirectional Google / Outlook Calendar incremental sync with webhook push.
- Webhook dispatch system with HMAC signature verification.

SPEAKER SCRIPT (4:45 - 5:00) — Conclusion & Next Steps:

“In conclusion, Slotbook solves scheduling reliability at its mathematical and transactional root. By shifting concurrency guarantees into PostgreSQL, automating outbox notifications, and enforcing strict timezone transformations, we have turned a fragile prototype into a resilient, enterprise-ready engine. Thank you for your time, and I look forward to your questions.”